

Mycelic + NeuralGraph: where the accuracy actually stands

Status report, 8 October 2026 · branch `claude/vnext-accuracy-70-percent-gsjg8m` (latest results commit `9d4711f`) · every number below is from a committed artifact unless marked otherwise

Bottom line

- **Simulator, 10,000 users:** Mycelic v4 finds **71%** of the hidden cross-org problems on seeds no one had read (0.713, 95% CI 0.678–0.750), up from 62.5%. The strongest centralised baseline (A2) finds **74%** on the same worlds, and **80%** vs Mycelic's 58% at 50,000 users. Centralised still wins on raw accuracy.
- **Real LLM agents (first run):** 429 employee agents, each a real Qwen3-1.7B call on its own notes, one 400-user world. Mycelic v4 finds **65%** (60% backed by real evidence); central triage on the same claims 45%. **The centralised A2 reader on the same model finds 87.5%**, with about a third of the model tokens (313k vs 919k). Mycelic accepts fewer decoys (0.35 vs 0.525) and moves no raw text. One world, 40 patterns: indicative, not a claim, but centralised wins this one plainly.
- **The biggest live failure is not discovery, it is negation.** The small model read 2 of 43 negated notes. Retractions vanish, so every stale-chain decoy got through (D5 acceptance 1.00 vs 0.20 in the simulator).
- **NeuralGraph memory (LoCoMo benchmark):** 72.2% with a lenient same-family judge (744 of 1,540 questions), 42.5% with a strict independent judge. Half the benchmark is unrun.
- **Your ingestion layer does not feed Tesseract.** The production memory server (`chat_memory`), the research service and the Tesseract retriever are three separate paths, and the only path that reaches Tesseract stamps every memory with the ingestion time instead of when it happened (section 4).

1. Every accuracy number we have

found = share of hidden cross-department patterns that end up in the 600-entry risk register. **Backed** = found with at least half of the pattern's links supported by its own evidence records (excludes coincidental matches). Decoys = planted look-alike patterns accepted.

System	Setting	found	backed	decoys	stale chains	Source
Mycelic v3 (frozen vNext)	sim 10k, seeds 15–29	0.625	–	0.375	0.767	quick_v4_heldout
Mycelic v4	sim 10k, seeds 15–29 (pre-registered)	0.713 [0.678, 0.750]	–	0.268	0.273	quick_v4_heldout
Mycelic v4	sim 10k, seeds 0–29 pooled	0.703 [0.678, 0.728]	–	0.261	0.243	quick_v4_heldout
A2 central long-context	sim 10k, seeds 0–29	0.743	–	–	–	quick_v4_heldout
Mycelic v4	sim 50k, seeds 0–2	0.583	–	0.253	0.27	quick_v4_heldout
A2 central long-context	sim 50k, seeds 0–2	0.797	–	–	–	quick_v4_heldout
Mycelic v4	live Qwen3-1.7B, 400 users, seed 702	0.650	0.600	0.350	1.00	stage_400_s702
Mycelic v3	live, same world	0.575	0.500	0.325	0.70	stage_400_s702
Central triage (B4)	live, same world	0.450	0.450	0.350	0.90	stage_400_s702
A2, same 1.7B model	live, same world	0.875	0.875	0.525	0.80	stage_400_s702_A2_matched

Live vs simulated operators on that same world: Mycelic v4 0.55 → 0.65, v3 0.60 → 0.575, central triage 0.575 → 0.45, A2 0.925 → 0.875. Simulator decoy acceptance (0.10–0.23 for the hierarchy) badly understates live decoy acceptance (0.33–0.35). Live model cost on this world: Mycelic 429 calls, 754k prompt + 165k output tokens, ~5.0 h of this CPU; A2 143 calls, 255k + 58k tokens, ~1.5 h. A2 reads only the notes its keyword prefilter flags as causal, which is exact on this generator's template text and would not be on real prose.

NeuralGraph memory on LoCoMo (conversational memory QA)

Judge	Questions	single	multi	temporal	open	overall
Gemma, lenient (same family as answerer)	744 (conv 1-5)	73.9	72.8	73.7	56.5	72.2
Qwen, lenient (independent)	584 (conv 1-4)	62.2	66.6	71.5	43.8	65.6
Qwen, strict (independent)	584 (conv 1-4)	33.3	41.8	60.8	6.2	42.5
substring match	584 (conv 1-4)	21.6	27.0	36.2	3.1	26.7

The same answer files score 13.5% to 64.9% depending only on the grader. The only gain that survives every judge is per-agent memory routing (+9 points single-hop). Conversations 6–10 (796 questions) have never been run.

Live extraction quality (16,233 notes, Qwen3-1.7B vs the simulator's assumed 7B model)

	simulator (7B-class operator)	real Qwen3-1.7B	
Notes that yield a claim	0.70	0.96	better
Invented claims per note	0.18	0.04	better
Right entity	0.84	0.99	better
Gold-pattern notes read exactly right	0.53	0.65	better
Wrong event type on causal notes	0.09	0.30	worse
Negated notes marked negated (smoke, 43 notes)	≈1.00	0.05	much worse
Echo (duplicate-report) detection from text	event id, 92% clean	1.00 pair recall	better

2. What is real and what is still simulated

Component	Simulator	Live run today
Employee agents reading their notes	coin flips on the hidden truth	real model call per agent
Duplicate/echo detection	hidden event id (8% noise)	computed from note text
Routing, sketch, triage, merging	code (by design)	same code
Answering kernel questions	filter on extracted claims	same (no fresh model read yet)
Kernel chain synthesis + verification	code with simulated checks	still simulated
Ranking the 600-entry register	learned ranker	same ranker (fitted on simulator)
Gold answers	evaluator only	evaluator only (tested)

So today's live numbers test whether discovery survives **real evidence extraction**, not whether a model can make the discovery leap itself.

3. Mycelic routing is not Dijkstra (yet)

You think of the routing as Dijkstra's algorithm. The code does something simpler. A kernel question descends the org tree; at each level it goes to at most 3 children, chosen greedily by $2 \times (\text{causal mentions of the entity}) + (\text{all mentions})$ in each child's index (research/mycelic/systems.py, Hierarchy.descend). There is no path cost, no priority queue and no global optimum. That matters: 19% of rare evidence records are never reached by a descent, and the descent fan-out experiments (3 → 5 → 7) were inside noise because a greedy rule spends its budget where mentions are loud, not where the missing link is likely. Question 3 below.

4. Does NeuralGraph ingestion produce memories Tesseract can absorb?

No. There are three disconnected memory paths, and the one that reaches Tesseract has a timestamp fault.

#	Finding	Where	Effect
F1	Production ingestion writes its own SQLite store (Memory: event_time, observed_at, superseded_by). Nothing reads it into Tesseract or into Mycelic.	NeuralGraph/chat_memory/*	Memories users create never reach Tesseract.

#	Finding	Where	Effect
F2	Research service ingests with correct message dates but never calls Tesseract's retriever (imports only a keyword helper).	research/retrieval/service.py: 61, :558	Correct timestamps, wrong retriever.
F3	The only Tesseract path (benchmark runner) stamps created_at = datetime.now(); the real date lives only in metadata['datetime'].	research/benchmarks/runner.py:651, :467	Every memory looks like it happened at ingestion.
F4	Tesseract resolves 'last year' from created_at.year first, metadata second.	tesseract.py:1250-1255	Relative years anchor to 2026, not 2022-24 (259 of 5,882 LoCoMo messages use relative time).
F5	No reference time is passed to retrieve(); recency term is a constant 0.5.	runner.py:769; tesseract.py:1233	Recency never ranks anything.
F6	Episode segmentation uses gaps between created_at values.	research/retrieval/hierarchy.py:367-376	With ingestion timestamps all gaps are ~0: episodes are meaningless on that path.
F7	Consolidation decay uses wall-clock age since update.	research/retrieval/consolidation.py:255	A bulk-imported history all looks equally fresh.
F8	Naive and timezone-aware datetimes are mixed (runner naive; NeuralNode default aware UTC).	runner.py:651; data_types.py	Latent crash the first time they are subtracted.
F9	Tesseract only searches message nodes; consolidated episode/topic/persona memories are never queried.	tesseract.py (no layer handling)	Consolidation work is invisible to retrieval.

Temporal synchronisation on the Mycelic side: the simulator assumes every site stamps events on one shared day clock; v4's biggest gain (same-day witness clusters) depends on it. Real sites have time zones, reporting delay and batch uploads. Not yet tested.

5. What is missing (the short list)

- **One temporal contract for the whole stack.** chat_memory already has the right model (event time with precision, observed time, superseded-by). Tesseract and Mycelic each invent their own and lose it.
- **Retraction as a first-class signal.** chat_memory has superseded_by and audited forgetting; Mycelic needs exactly that and currently relies on a small model noticing the word 'not'.
- **The bridge between personal memory and the enterprise hierarchy.** A Mycelic employee agent should be that employee's NeuralGraph memory. Today they are different codebases with different data models.
- **A reason to distribute for accuracy.** In the benchmark every note is self-contained, so a central reader with all the text has strictly more information. Distribution can only tie or lose on accuracy unless local agents know something the centre cannot.
- **A model at the kernel.** Discovery decisions are still code; the live run cannot yet tell us whether an LLM kernel would find more or hallucinate more.

6. Questions for the chief research engineer

Ordered by expected impact per hour of work. Each says why it matters, the evidence, and the cheapest test (all free on local models unless noted).

Q1. Should every Mycelic employee agent simply be a NeuralGraph chat_memory instance?

chat_memory already extracts structured memories with event time, confidence and superseded_by. Mapping Memory → Mycelic claim (subject → entity, kind → event type, event_time → day, superseded → retraction) is an adapter, not a redesign. It would give Mycelic the retraction signal the live run is missing, and give NeuralGraph its enterprise layer.

Test: adapter + rerun the 400-agent world with claims from chat_memory's extraction prompt; watch stale-chain decoys (1.00 today).

Q2. Can we adopt one bitemporal memory contract everywhere (valid time vs recorded time)?

Findings F2–F8 are all the same bug: 'when it happened', 'when we heard it' and 'when we stored it' are conflated. A retraction then becomes 'valid until', which is exactly what stale-chain detection needs.

Test: set created_at from the message date in the benchmark runner, prefer metadata time in Tesseract, rerun LoCoMo temporal questions (321) under the strict judge.

Q3. Should routing become real best-first search: Dijkstra/A* with a learned 'evidence likely here' heuristic?

Today it is a greedy 3-child descent by mention counts (section 3); 19% of rare facets are never reached and widening the fan-out did nothing. A priority-queue search whose edge cost is communication and whose heuristic is the sketch/Bloom estimate of the missing link spends the same budget where the answer is.

Test: same question budget, A* vs greedy descent, measure descent reach and found on tune seeds.

Q4. Should edge agents get a tiny verification tool: 'is this note denying or retracting X?'

The live model read 2 of 43 negations; every stale-chain decoy was accepted. A rule-based negation check, or one yes/no follow-up call per flagged note, is the cheapest high-impact fix we have found. It is also a distributed advantage: each agent re-checks its own 30 notes; a central reader cannot afford to re-read millions.

Test: add the tool, rerun the same 429 agents from cache for everything else; compare D5 and found.

Q5. Instead of ranking 3,000 candidates into 600 slots, can the kernel falsify them?

Real cross-org problems propagate region to region; background noise does not. Ask the holders of a candidate's predicted next link in another region for yes/no + counts (no raw text). A wrong prediction kills the candidate. This attacks the register cut, the single biggest loss in the simulator, and coincidental matches.

Test: one falsification round before the cut on tune seeds; track found, backed found and decoys.

Q6. Where is Mycelic supposed to beat centralised on accuracy, not just privacy?

In this benchmark a central reader with all text has strictly more information, so A2 wins (0.74 vs 0.71 at 10k, 0.80 vs 0.58 at 50k). Real enterprises have context only the local agent has: abbreviations, history, who 'the vendor' is. If that is the thesis, the generator must contain context-dependent notes, or we will never measure the advantage.

Test: add notes resolvable only with the author's earlier notes; compare Mycelic vs A2 at equal compute.

Q7. Is final recall even the right metric, or is it time-to-detection?

Continual discovery is the point of the architecture: incremental local agents can flag an emerging pattern while a central batch reader is still re-reading. The benchmark only scores the end state.

Test: stream the 180 simulated days, score the day each pattern first enters the register.

Q8. How do we keep the clocks honest across sites?

v4's main gain is same-day witness clusters. With real time zones, reporting delays and batch uploads, those clusters smear. Estimating per-site offsets from shared events (NTP over evidence) is cheap.

Test: add per-site clock offset and delay to the generator; rerun v4 vs A2.

Q9. What should the kernel model do when we add one: verify, or generate hypotheses?

Synthesis is still code with simulated checks. A model that verifies candidates is safer; one that proposes chains could find what the 7 templates miss, but may hallucinate. We need a decision before building it.

Test: LLM judge on the top 600 only (cheap), then on all candidates.

Q10. Which NeuralGraph number do we stand behind?

72.2% (lenient, same family) and 42.5% (strict, independent) describe the same answers. Commit to the strict independent judge and finish conversations 6–10 before any claim leaves the team.

Cost: about 2 hours of local model time; scripts exist (n5_rescore, capstone runner).

Q11. Should Tesseract's fusion be rank-based instead of near-uniform weights?

The temporal store dominates the top 10 for every question type, and 18 gold messages are dropped by filters before ranking. Rank fusion across stores was the one untested combination with signal.

Test: reciprocal-rank fusion of the four stores on the 1,219-question retrieval set.

7. What I would do next (free, local)

- Finish the A2 live comparison (running) and add 2 more live worlds so the live table is not one world.
- Fix F3/F4/F5 (three lines) and rerun LoCoMo temporal questions under the strict judge.
- Add the negation tool (Q4) and rerun the 429 agents (cached calls replay; only the new check costs time).
- Run 2,000 and 10,000 agents on the MacBook Air (Metal) with `research/mycelic/live/mac_setup.sh`; this container needs ~47 h and ~234 h.
- Resume the v5 protocol (fair comparators, sealed final seeds 3000–3029) once helper agents are available again (Oct 12).

Reproduce

```
python3 -m research.mycelic.live.run --stage 400 # live agents (needs a llama-server)
python3 -m research.mycelic.arm_paired ... # simulator paired runs
python3 -m unittest research.mycelic.test_mycelic research.mycelic.test_leakage
research.mycelic.live.test_live
artifacts: research/mycelic/artifacts/quick_v4_heldout.jsonl, research/mycelic/artifacts/live/
```